

Plataforma de monitoreo inteligente – Grupo Cordillera.

Fullstack

Integrantes: Darlynee Zamorano - Maria Paz Silva

Sección: 004D.

Profesora: Viviana Poblete.

Fecha de entrega: 18/06/2026

Índice

1. Contextualización del caso	3
2. Selección de patrones de arquitectura	4
2.1 Patrones de arquitectura utilizados	4
2.2 Tecnologías y herramientas	4
2.3 Cómo funciona Spring Boot	4
2.4 Estrategia de gestión de versiones (Git)	5
3. Comunicación entre microservicios	6
3.1 Funcionamiento de la comunicación	6
3.2 Tipo de comunicación	7
3.3 Tecnologías de comunicación entre microservicios	7
3.4 Justificación de la arquitectura	7
3.5 Modelo de Persistencia y Estructura de Datos	7
4. Arquitectura de contenedores (Docker y Kubernetes)	9
4.1 ¿Cómo funciona Docker?	9
4.2 Componentes de Docker	9
4.3 ¿Cómo funciona Kubernetes?	9
5. Desarrollo Frontend y Experiencia de Usuario (UX)	11
5.1. Módulo Operativo (Vista de Usuario / Cajero)	11
5.2. Módulo Analítico y Geolocalización (Vista de Administrador)	11
5.3. Despliegue del Frontend mediante Nginx	11
6. Aseguramiento de Calidad y Pruebas Unitarias (Testing)	12
6.1. Pruebas de la Capa de Control (VentaControllerTest)	12
6.2. Pruebas de la Capa de Negocio (VentaServiceImplTest)	12
6.3 Mejoras a futuro	12
7. Conclusión	13
8. Bibliografía	14

1. Contextualización del caso

Grupo Cordillera es una empresa del sector retail dedicada a la comercialización de productos para el hogar y tecnología, con presencia en diversas ciudades del país. Debido al crecimiento, la organización ha incorporado distintos sistemas tecnológicos, tales como plataformas de venta, sistemas de gestión de inventario, herramientas financieras y sistemas de atención al cliente. Sin embargo, estos sistemas operan de manera independiente, lo que ha generado una dispersión de la información y dificultades para su integración.

Esto ha traído como consecuencia que la empresa enfrente problemas en la generación de reportes, ya que los datos deben ser recopilados manualmente desde diferentes fuentes, procesados y validados antes de ser utilizados. Este proceso no solo es lento, sino que también aumenta el riesgo de errores y limita la disponibilidad de información en tiempo real. En este contexto, surge la necesidad de implementar una solución basada en arquitectura de microservicios que permita centralizar los datos, automatizar procesos y mejorar la toma de decisiones mediante el monitoreo eficiente del desempeño organizacional.

2. Selección de patrones de arquitectura

Para dar solución a la problemática presentada por el Grupo Cordillera, se propone una arquitectura basada en microservicios, la cual permite dividir el sistema en componentes independientes, facilitando su mantenimiento, escalabilidad y evolución.

Esta arquitectura se apoya en distintos patrones de diseño que permiten estructurar de manera eficiente el sistema y responder a los requerimientos del cliente.

2.1 Patrones de arquitectura utilizados

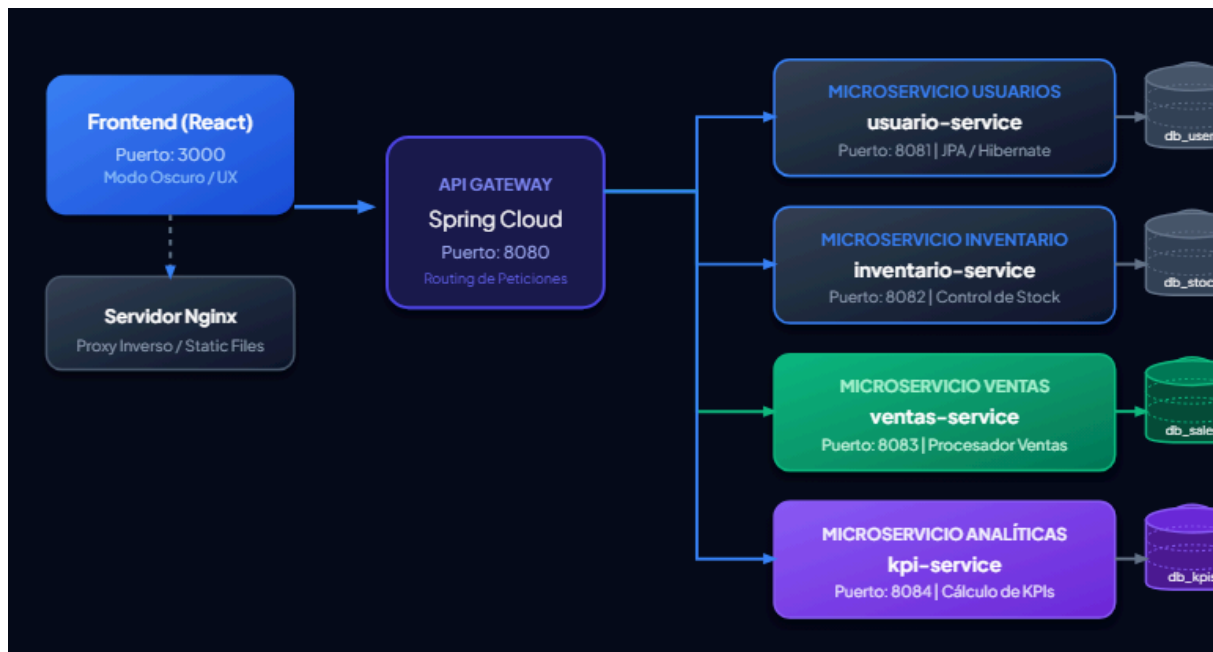
- El sistema para Grupo Cordillera se estructura bajo una arquitectura de microservicios, donde las funcionalidades se han desglosado en cuatro servicios independientes y especializados: **Usuarios**, **Ventas**, **Inventario** y **KPI**. Cada microservicio cuenta con su propia base de datos dedicada, garantizando el desacoplamiento y la integridad de los datos mediante el patrón *Database per Service*. El acceso centralizado se gestiona a través de un **API Gateway**, el cual actúa como el único punto de entrada para el frontend, encargándose del enrutamiento, la autenticación mediante tokens JWT y la seguridad global del sistema.

2.2 Tecnologías y herramientas

- Backend: Spring Boot, RabbitMQ, Spring Cloud Gateway y FeignClient
- Frontend: React
- Base de datos: MySQL
- Control de versiones: Git

2.3 Cómo funciona Spring Boot

Spring Boot es un framework de desarrollo basado en Java que permite crear aplicaciones backend de forma rápida y eficiente. Su principal ventaja es que simplifica la configuración del proyecto, permitiendo desarrollar microservicios de manera modular, independiente y escalable. Además, facilita la integración con base de datos mediante tecnologías como JPA, lo que permite gestionar la persistencia de datos de forma estructurada



2.4 Estrategia de gestión de versiones (Git)

Se utilizó la estrategia GitHub Flow, la cual es ideal para ciclos de entrega continua. Consiste en una rama main que siempre contiene código desplegable y ramas feature/ para cada funcionalidad. Esta elección simplificó la coordinación del equipo, permitiendo integrar cambios rápidamente y realizar *code reviews* antes de fusionar, minimizando conflictos y asegurando la estabilidad del código base.

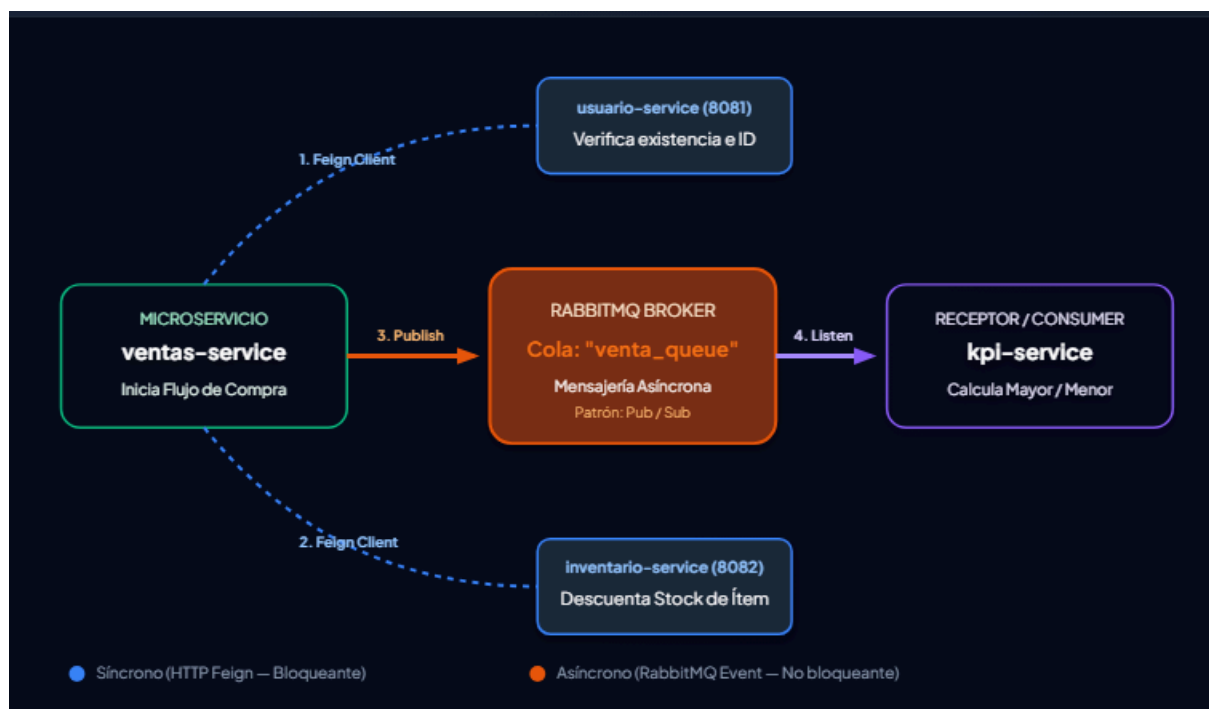
3. Comunicación entre microservicios

La comunicación entre los microservicios se realiza a través de un API Gateway, el cual actúa como el punto de entrada único para todas las solicitudes del cliente. Este componente recibe las peticiones desde el frontend y se encarga de redirigirlas hacia el microservicio correspondiente, según la funcionalidad requerida.

Esta arquitectura centraliza la gestión de las solicitudes, mejora la seguridad y simplifica la interacción entre el cliente y los servicios internos del sistema.

3.1 Funcionamiento de la comunicación

La comunicación es híbrida para garantizar la eficiencia. Por un lado, utilizamos **comunicación síncrona** mediante *OpenFeign* para las solicitudes que requieren respuesta inmediata (como la validación de usuarios o consulta de productos). Por otro lado, implementamos **comunicación asíncrona** mediante **RabbitMQ** para eventos críticos, como cuando el servicio de *Ventas* notifica al servicio de *Inventario* para descontar stock o al servicio de *KPI* para actualizar las métricas en tiempo real, garantizando que el sistema sea resiliente ante picos de demanda.



3.2 Tipo de comunicación

La comunicación entre los microservicios se realiza mediante APIs REST, utilizando el formato JSON para el intercambio de datos. Este tipo de comunicación permite que los servicios sean independientes, flexibles y fácilmente integrables.

3.3 Tecnologías de comunicación entre microservicios

Además del uso de API Gateway como punto de entrada, la comunicación interna entre los microservicios se realiza principalmente mediante APIs REST.

Para optimizar esta comunicación, se utiliza WebClient, el cual permite realizar llamadas HTTP de manera reactiva y no bloqueante, mejorando el rendimiento del sistema en escenarios de alta concurrencia.

Adicionalmente, se considera el uso de mensajería asíncrona mediante RabbitMQ, lo que permite que los microservicios intercambien información sin depender directamente unos de otros, favoreciendo el desacoplamiento y la escalabilidad del sistema.

En conjunto, estas tecnologías permiten implementar una comunicación eficiente, flexible y alineada a los requerimientos del sistema.

3.4 Justificación de la arquitectura

La implementación del API Gateway permite organizar de manera eficiente el flujo de información dentro del sistema, facilitando la gestión de solicitudes, el control de acceso y la seguridad. Además, esta arquitectura mejora la eficiencia técnica y operativa, permitiendo un sistema más ordenado, escalable y fácil de mantener.

3.5 Modelo de Persistencia y Estructura de Datos

Para garantizar la persistencia de datos y la coherencia del sistema, cada microservicio implementa el patrón *Repository* utilizando *JPA/Hibernate* sobre bases de datos MySQL independientes. En el entorno de despliegue con Docker, se han configurado **volúmenes persistentes** (mapeos locales), asegurando que la información de los productos, usuarios y ventas no se pierda al reiniciar o actualizar los contenedores del sistema

- Estructura del Microservicio de Ventas: Implementa validaciones estrictas mediante `jakarta.validation` para el procesamiento de compras. El objeto DTO

`VentaSolicitud` captura los identificadores correlativos, montos exactos mediante el tipo de dato de alta precisión matemática `BigDecimal` y la modalidad de pago.

-
- Control Estricto de Métodos de Pago:** El flujo financiero no utiliza cadenas de texto libres para los pagos. Se definió un tipo enumerado (`Enum`) denominado `MetodoPago` con los valores fijos del negocio: `EFECTIVO`, `TRANSFERENCIA`, `TARJETA` y `CREDITO`. Esto garantiza que la base de datos relacional MySQL mantenga la integridad referencial y bloquee transacciones malformadas en el origen.

4. Arquitectura de contenedores (Docker y Kubernetes)

Para el despliegue del sistema, se propone el uso de contenedores mediante Docker, lo que permite empaquetar cada microservicio junto con todas sus dependencias necesarias para su ejecución. Esto asegura que el sistema pueda ejecutarse de manera consistente en distintos entornos, independientemente del dispositivo o sistema operativo utilizado.

Además, se considera el uso de Kubernetes como herramienta de orquestación de contenedores, permitiendo gestionar múltiples servicios de forma automatizada, escalable y eficiente.

4.1 ¿Cómo funciona Docker?

Docker es una plataforma que permite crear, ejecutar y gestionar contenedores. Cada contenedor incluye todo lo necesario para ejecutar una aplicación, como el código, librerías y configuraciones. Esto facilita el despliegue del sistema y asegura que funcione correctamente en cualquier entorno.

4.2 Componentes de Docker

- Imagen: plantilla que contiene la aplicación
- Contenedor: instancia en ejecución de la imagen
- Dockerfile: archivo donde se define cómo construir la imagen

4.3 ¿Cómo funciona Kubernetes?

Kubernetes es una herramienta que permite administrar múltiples contenedores, automatizando tareas como el despliegue, escalado y monitoreo de las aplicaciones. Esto es especialmente útil en arquitecturas de microservicios, donde existen múltiples componentes funcionando de manera simultánea.

4.4 Justificación de la arquitectura

El uso de Docker y Kubernetes facilita mejorar la eficiencia técnica del sistema, facilitando su despliegue, mantenimiento y escalabilidad. Además, contribuye a la sostenibilidad del sistema, ya que permite optimizar recursos y adaptarse a futuras necesidades del negocio.



5. Desarrollo Frontend y Experiencia de Usuario (UX)

La interfaz de usuario de la plataforma del Grupo Cordillera fue desarrollada utilizando React, implementando una arquitectura reactiva basada en componentes modulares. El flujo de datos en tiempo real se gestiona a través del Hook ``useState`` para el control de estados locales, y el Hook ``useEffect`` para sincronizar y consumir de forma asíncrona los endpoints expuestos por el API Gateway. La distribución visual utiliza componentes de diseño estructurados en mallas (****Grid****), garantizando una visualización responsiva.

Para resguardar la confidencialidad de la información gerencial y optimizar la operación en las tiendas, se implementó un Control de Acceso Basado en Roles (RBAC), dividiendo la experiencia de usuario en dos vistas independientes:

5.1. Módulo Operativo (Vista de Usuario / Cajero)

Diseñado exclusivamente para la velocidad transaccional del personal en caja. Es un formulario interactivo limpio que permite buscar productos, ingresar cantidades y seleccionar el método de pago mediante componentes desplegables vinculados directamente al Enum ``MetodoPago``. No incluye gráficos ni acceso a métricas globales de la empresa, operando de manera directa con el ``ventas-service``.

5.2. Módulo Analítico y Geolocalización (Vista de Administrador)

Panel de control gerencial de alta fidelidad diseñado para el monitoreo del rendimiento comercial del Grupo Cordillera. Este panel consume los datos calculados de manera asíncrona por el ``kpi-service`` y expone dos componentes críticos:

- **Gráficos Dinámicos con Recharts:** Despliega gráficos de barras interactivos para visualizar los productos más vendidos del inventario y gráficos de torta analíticos que segmentan la proporción de ingresos según el método de pago utilizado (Efectivo vs. Transferencia vs. Tarjeta).
- **Biblioteca de Mapas con Leaflet:** Integra un mapa interactivo geolocalizado que renderiza las coordenadas de las diversas sucursales del país, pintando indicadores visuales dinámicos sobre cuáles tiendas físicas registran el mayor volumen de facturación monetaria en tiempo real.

5.3. Despliegue del Frontend mediante Nginx

Para el entorno de producción, la aplicación de React se compila en archivos estáticos optimizados y se despliega utilizando Nginx como servidor web de alto rendimiento. Nginx actúa adicionalmente como proxy inverso, protegiendo las IP internas de los microservicios, gestionando las directivas de CORS (Cross-Origin Resource Sharing) y balanceando la carga de las solicitudes HTTP entrantes.

6. Aseguramiento de Calidad y Pruebas Unitarias (Testing)

Siguiendo las métricas de calidad exigidas en el desarrollo de software moderno, el microservicio principal `ventas-service` fue sometido a un riguroso proceso de pruebas automatizadas utilizando JUnit 5 y Mockito, alcanzando una cobertura lógica limpia que certifica la estabilidad del sistema antes de su empaquetamiento en Docker.

6.1. Pruebas de la Capa de Control (VentaControllerTest)

Se implementaron pruebas mediante `MockMvc` para simular peticiones HTTP concurrentes directamente sobre los endpoints del controlador, verificando los códigos de estado de la arquitectura REST:

- Verificación de Creación (201 Created): Comprueba que al enviar un payload JSON válido con un método de pago correcto (ej: `MetodoPago.TARJETA`), el controlador procese la venta y responda con el identificador generado.
- Control de Solicitudes Malformadas (400 Bad Request): Valida que los filtros de Spring intercepten y bloqueen transacciones que omitan campos obligatorios.

6.2. Pruebas de la Capa de Negocio (VentaServiceImplTest)

Dado que el microservicio de ventas depende de datos alojados en otros contenedores, se utilizó el mecanismo de aislamiento de Mockito (`@Mock` y `@InjectMocks`) para evadir llamadas de red reales durante la fase de compilación (`mvn test`):

- Aislamiento de Microservicios Perimetrales:** Se simuló las llamadas síncronas de OpenFeign hacia el microservicio de usuarios (`UsuarioClient`) retornando DTOs controlados, y hacia el servicio de inventario (`InventarioClient`), emulando adecuadamente las envolturas genéricas `ResponseEntity<?>` que Spring Boot utiliza para la comunicación inter-servicio.
- Verificación de Mensajería Asíncrona: Se auditó mediante el método `verify()` de Mockito que, tras procesar una transacción exitosa, el sistema invoque exactamente una vez al componente `RabbitTemplate`, garantizando que el mensaje de actualización de reportes viaje correctamente hacia las colas de RabbitMQ.

6.3 Mejoras a futuro

Aunque la arquitectura de microservicios resolvió la dispersión de datos, enfrentamos el desafío de la latencia en las llamadas síncronas. Como mejora, para futuras implementaciones, se propone la implementación de un caché distribuido como **Redis** para evitar llamadas innecesarias al inventario-service, optimizando aún más el rendimiento global del sistema.

7. Conclusión

El desarrollo de este proyecto nos permitió entender de mejor manera la importancia de una buena arquitectura de software para resolver problemas reales, como los que enfrenta el Grupo Cordillera. A través del análisis del caso, pudimos ver cómo una mala integración de sistemas puede afectar directamente la gestión de la información y la toma de decisiones dentro de una organización. La arquitectura basada en contenedores Docker contribuye a la sostenibilidad tecnológica al maximizar la densidad de cómputo en los servidores, reduciendo el consumo energético. En cuanto a la ética, la privacidad se gestiona mediante tokens JWT y cifrado, asegurando que solo los usuarios autorizados accedan a la información sensible de ventas.

También comprendimos el rol que cumplen distintas tecnologías como Spring Boot, API Gateway y Docker, y cómo estas se complementan para construir una solución más eficiente. Esto nos ayudó a tener una visión más clara de cómo se estructuran los sistemas modernos.

De manera personal, este proyecto creo que nos permitió entender que no se trata solo de programar, sino de pensar bien cómo se construyen las soluciones desde el inicio, considerando tanto lo técnico como las necesidades del negocio.

8. Bibliografía

Spring Boot. (2024). <https://spring.io>

Docker. (2024). <https://www.docker.com>

Kubernetes. (2024). <https://kubernetes.io>

React. (2024). <https://react.dev>

MySQL. (2024). <https://www.mysql.com>

Newman, S. (2015). Building Microservices. O'Reilly Media.